

```
# Complete Explanation of D-G05-Final-Project.Rmd
```

```
## 📄 Project Overview
```

This R Markdown file implements a **comparative classification analysis** on stroke prediction** using machine learning. It processes healthcare data and compares three classification models (Logistic Regression, Random Forest, and SVM) to predict stroke occurrence.

```
---
```

```
## 💡 YAML Front Matter (Lines 1-6)
```

```
```yaml
```

```

```

```
title: "A Comparative Classification Analysis on Stroke Prediction"
```

```
author: "Niloy Paul, Mithila Chowdhury Medha, Anik Mistry"
```

```
date: "`r Sys.Date()`"
```

```
output: html_document
```

```

```

```
```
```

```
**Explanation:**
```

- ****title****: Sets the main heading of the HTML output document
- ****author****: Lists the three project authors
- ****date****: ``r Sys.Date()`` dynamically inserts today's date when the document is knitted
- ****output****: Specifies that the output should be an HTML document

```
---
```

```
## CHUNK 1: Setup (Lines 7-17)
```

```
```r
```

```
knitr::opts_chunk$set(
 fig.align = "center",
 fig.height = 5,
 fig.width = 9
)
```

```
set.seed(42)
```

```
```
```

```
**What it does:** Configures global settings for all code chunks in the document
```

```
**Parameters explained:**
```

- ``knitr::opts_chunk$set()``: Sets default options for all R code chunks
 - ``fig.align = "center"``: All plots will be centered in the output
 - ``fig.height = 5``: Height of all figures is 5 inches
 - ``fig.width = 9``: Width of all figures is 9 inches
- ``set.seed(42)``: Ensures reproducibility by fixing the random number generator seed to 42
 - This means any random sampling/splitting will produce identical results every time

CHUNK 2: Load Libraries (Lines 33-61)

```
```r
required_packages <- c(
 "tidyverse", # Data manipulation and visualization
 "ggplot2", # Advanced plotting
 "corrplot", # Correlation matrix visualization
 "gridExtra", # Multi-panel plots
 "caret", # Unified ML training and evaluation framework
 "randomForest", # Random Forest classifier
 "xgboost", # XGBoost classifier
 "e1071", # Support Vector Machine
 "ROSE", # Random Over-Sampling Examples (class imbalance)
 "knitr", # Table formatting
 "kableExtra", # Enhanced table styling
 "scales", # Axis formatting in ggplot
 "pROC", # ROC curve analysis
 "ggcorrplot" # Correlation plot with ggplot2 style
)
```
```

****What it does:**** Defines all required packages as a character vector

****Package descriptions:****

| Package | Purpose |
|-----------------------------|---|
| <code>`tidyverse`</code> | Meta-package with data manipulation tools (dplyr, ggplot2, tidyr) |
| <code>`ggplot2`</code> | Grammar of graphics for advanced plotting |
| <code>`corrplot`</code> | Visualizing correlation matrices |
| <code>`gridExtra`</code> | Combining multiple plots into single figure |
| <code>`caret`</code> | Classification and Regression Training framework |
| <code>`randomForest`</code> | Random Forest algorithm implementation |
| <code>`xgboost`</code> | Extreme Gradient Boosting (not used in this project) |
| <code>`e1071`</code> | Support Vector Machines and other ML algorithms |
| <code>`ROSE`</code> | Handles imbalanced datasets through oversampling |
| <code>`knitr`</code> | Converts R Markdown to various formats |
| <code>`kableExtra`</code> | Enhanced table styling and formatting |
| <code>`scales`</code> | Helps format axis labels (percentages, etc.) |
| <code>`pROC`</code> | ROC curve and AUC analysis |
| <code>`ggcorrplot`</code> | ggplot2-style correlation plots |

****Auto-installation logic:****

```
```r
new_packages <- required_packages[!(required_packages %in%
installed.packages()[,"Package"])]
```
```

- ``installed.packages()[,"Package"]`` gets all installed package names
- ``!(required_packages %in% ...)`` finds packages NOT in the installed list
- ``install.packages()`` installs missing packages from CRAN

```

```r
invisible(lapply(required_packages, library, character.only = TRUE))
```

- `lapply()` applies the `library()` function to each package
- `character.only = TRUE` allows package names as strings
- `invisible()` suppresses the output

---

## CHUNK 3: Dataset Import (Lines 66-85)

```r
file_id <- "1Z28Czhbfn2zglnz9vAg0TYgmIrQeHRxE"
gdrive_url <- paste0("https://drive.google.com/uc?export=download&id=",
file_id)
temp_file <- tempfile(fileext = ".csv")
```

**What it does:** Downloads the dataset from Google Drive

**Step-by-step breakdown:**

1. **`file_id`**: Unique identifier for the Google Drive file
2. **`gdrive_url`**: Constructs the download URL
   - `paste0()` concatenates strings without spaces
   - Google Drive download URL format:
     `https://drive.google.com/uc?export=download&id={file_id}`
3. **`temp_file`**: Creates a temporary file path
   - `tempfile()` creates a unique temporary file location
   - `fileext = ".csv"` specifies file extension

**Download with error handling:**

```r
```r
tryCatch({
  download.file(gdrive_url, destfile = temp_file, mode = "wb", quiet =
TRUE)
  stroke_raw <- read.csv(temp_file, stringsAsFactors = FALSE, na.strings
= c("N/A", "NA", ""))
  cat("Dataset loaded successfully from Google Drive.\n")
}, error = function(e) {
  stop("Failed to load dataset from Google Drive...")
})
```

Parameters explained:

- `download.file()`:
 - `destfile = temp_file`: Where to save the downloaded file
 - `mode = "wb"`: Write binary mode (safe for all file types)
 - `quiet = TRUE`: Suppress progress messages

- `read.csv()`:
 - `stringsAsFactors = FALSE`: Keeps text columns as strings (not
factors)

```

- ``na.strings = c("N/A", "NA", "")``: Treats these values as missing data

- ``tryCatch()``: Error handling
  - First argument: Code to try
  - ``error = function(e)``: Code to run if error occurs

---

## CHUNK 4: Dataset Description (Lines 90-95)

```
```\r
cat("Dataset Dimensions:", nrow(stroke_raw), "rows x", ncol(stroke_raw),
"columns\n\n")
str(stroke_raw)
```
```

**\*\*What it does:\*\*** Displays dataset structure

- ``cat()``: Prints text output with string concatenation
- ``nrow()``: Returns number of rows
- ``ncol()``: Returns number of columns
- ``str()``: Shows structure (column names, types, first few values)

---

## CHUNK 5: Summary Statistics (Lines 102-105)

```
```\r
summary(stroke_raw)
```
```

**\*\*What it does:\*\*** Shows statistical summaries for all variables

- For numeric columns: min, 25%, median, 75%, max, mean, NA count
- For character columns: shows the class and length

---

## CHUNK 6: Missing Values Detection (Lines 110-128)

```
```\r
missing_counts <- colSums(is.na(stroke_raw))
missing_pct <- round(missing_counts / nrow(stroke_raw) * 100, 2)
```
```

**\*\*Step-by-step:\*\***

1. **\*\*`colSums(is.na(stroke\_raw))`\*\***:
  - ``is.na()`` creates a logical matrix (TRUE for missing, FALSE otherwise)
  - ``colSums()`` sums each column (TRUE counts as 1)
  - Result: Count of missing values per column
2. **\*\*`missing\_pct`\*\***: Calculates percentage missing

```

- `missing_counts / nrow(stroke_raw)` gives fraction missing
- `* 100` converts to percentage
- `round(..., 2)` rounds to 2 decimal places

Creating the display table:
```r
missing_df <- data.frame(
  Column = names(missing_counts),
  Missing_Count = missing_counts,
  Missing_Percent = missing_pct
) %>% filter(Missing_Count > 0)
```

- Creates a data frame with three columns
- `%>%` pipes to `filter()` which removes rows where count = 0
- `kable()` and `kable_styling()` format as HTML table

CHUNK 7: Target Variable Distribution (Lines 133-152)

```r
stroke_counts <- stroke_raw %>%
  count(stroke) %>%
  mutate(
    label = ifelse(stroke == 1, "Stroke (1)", "No Stroke (0)"),
    percent = round(n / sum(n) * 100, 1)
  )
```

Pipeline breakdown:

1. **`count(stroke)`**: Counts observations per stroke class
 - Creates columns: `stroke` and `n` (count)

2. **`mutate()`**: Adds new columns
 - `ifelse()`: Creates labels (if stroke == 1, then "Stroke (1)", else "No Stroke (0)")
 - Calculates percentage: `n / sum(n) * 100`

Plotting:
```r
ggplot(stroke_counts, aes(x = label, y = n, fill = label))
```

- **`aes()`** (aesthetics): Maps data to visual properties
 - `x = label`: X-axis shows stroke labels
 - `y = n`: Y-axis shows counts
 - `fill = label`: Bar colors by label

```r
geom_bar(stat = "identity", width = 0.5, color = "white")
```

- `stat = "identity"`: Uses y-values directly (not counting observations)

```

```

- `width = 0.5`: Bar width is half default
- `color = "white"`: White bar borders

```r
geom_text(aes(label = paste0(n, "\n(", percent, "%)")), vjust = -0.3,
size = 4.5)
```

- Adds text labels on bars
- `paste0()`: Concatenates count and percentage
- `vjust = -0.3`: Vertical adjustment (above bar)

```r
scale_fill_manual(values = c("#2196F3", "#F44336"))
```

- Manually sets colors: Blue (#2196F3) and Red (#F44336)

```r
ylim(0, max(stroke_counts$n) * 1.15)
```

- Sets y-axis limits with 15% extra space at top

```

## CHUNK 8: Numerical Features Distribution (Lines 158-176)

```

```r
num_vars <- c("age", "avg_glucose_level", "bmi")

plots <- lapply(num_vars, function(var) {
  ggplot(stroke_raw %>% filter(!is.na(.data[[var]])),
    aes(x = .data[[var]], fill = as.factor(stroke))) +
    geom_histogram(bins = 30, alpha = 0.75, position = "identity") +
    ...
})

do.call(grid.arrange, c(plots, ncol = 3))
```

```

**\*\*Explanation:\*\***

- \*\*`lapply()`\*\***: Creates a plot for each numerical variable
  - `function(var)`**: Anonymous function receives each variable name
  - `.data[[var]]`**: Non-standard evaluation using **`.data`** pronoun (tidyverse)
- \*\*`filter(!is.na())`\*\***: Removes rows with missing values
- \*\*`geom\_histogram()`\*\***: Creates histogram
  - `bins = 30`**: Divides data into 30 bins
  - `alpha = 0.75`**: 75% opacity (allows overlapping to show through)
  - `position = "identity"`**: Overlaps histograms (not stacked)
- \*\*`do.call(grid.arrange, c(plots, ncol = 3))`\*\***:
  - `grid.arrange()`** combines multiple plots

- ``c(plots, ncol = 3)`` converts list of plots into arguments
- ``ncol = 3``: Arranges in 3 columns

---

## CHUNK 9: Categorical Features Distribution (Lines 182-202)

```
```\r
cat_vars <- c("gender", "ever_married", "work_type", "Residence_type",
"smoking_status")

cat_plots <- lapply(cat_vars, function(var) {
  ggplot(...) +
    geom_bar(position = "fill", alpha = 0.85) +
    scale_y_continuous(labels = percent_format()) +
    ...
})
```\r
```

**\*\*Key differences from numerical plots:\*\***

- **\*\*`geom\_bar(position = "fill")`\*\***: Stacked 100% bar chart
  - Shows proportions of each stroke class by category
  - Each bar totals to 100%
- **\*\*`scale\_y\_continuous(labels = percent\_format())`\*\***:
  - ``percent_format()`` from ``scales`` package
  - Formats y-axis as percentages (0.5 → 50%)
- **\*\*`theme(axis.text.x = element\_text(angle = 30, hjust = 1))`\*\***:
  - Rotates x-axis labels 30 degrees
  - ``hjust = 1``: Right-aligns labels for readability

---

## CHUNK 10: Scatter Plot (Lines 207-222)

```
```\r
ggplot(stroke_raw %>% filter(!is.na(bmi)),
  aes(x = age, y = avg_glucose_level,
    color = as.factor(stroke), alpha = as.factor(stroke))) +
  geom_point(size = 1.8) +
  scale_color_manual(values = c("0" = "#90CAF9", "1" = "#C62828")) +
  scale_alpha_manual(values = c("0" = 0.4, "1" = 0.9), guide = "none")
```\r
```

**\*\*Explanation:\*\***

- **\*\*`geom\_point()`\*\***: Creates scatter plot
  - ``size = 1.8``: Point size (default is 2)
- **\*\*`scale\_alpha\_manual()`\*\***: Sets opacity by stroke class
  - "0" (No Stroke): 40% opacity (faded)
  - "1" (Stroke): 90% opacity (prominent)

- `guide = "none"`: Removes alpha legend

This visualization shows how age and glucose relate to stroke occurrence, with stroke cases more visible.

---

## CHUNK 11: Outlier Detection (Lines 227-248)

```
```r
stroke_long <- stroke_raw %>%
  select(age, avg_glucose_level, bmi, stroke) %>%
  pivot_longer(cols = c(age, avg_glucose_level, bmi),
               names_to = "variable", values_to = "value") %>%
  filter(!is.na(value))
```
```

**\*\*Data transformation:\*\***

1. **\*\*`select()`\*\***: Keeps only relevant columns
2. **\*\*`pivot\_longer()`\*\***: Converts wide to long format
  - **\*\*Before\*\***: Separate columns for age, glucose, bmi
  - **\*\*After\*\***: One "variable" column and one "value" column
  - Example:

```
Before: age=50, glucose=120, bmi=25
```

```
After:
```

```
variable=age, value=50
variable=glucose, value=120
variable=bmi, value=25
```
```

3. ****`filter(!is.na(value))`****: Removes missing values

****Boxplot visualization:****

```
```r
ggplot(stroke_long, aes(x = as.factor(stroke), y = value, fill =
as.factor(stroke))) +
 geom_boxplot(outlier.color = "red", outlier.alpha = 0.4, alpha = 0.7,
width = 0.5) +
 facet_wrap(~variable, scales = "free_y")
```
```

- ****`geom\_boxplot()`****: Shows median, quartiles, and outliers
- ****`outlier.color = "red"`****: Outliers shown in red
- ****`outlier.alpha = 0.4`****: Semi-transparent outliers
- ****`facet\_wrap(~variable)`****: Creates separate plot for each variable
- ****`scales = "free\_y"`****: Each subplot has independent y-axis

CHUNK 12: Correlation Analysis (Lines 254-272)


```

```r
corr_data <- stroke_raw %>%
 select(age, avg_glucose_level, bmi, hypertension, heart_disease,
stroke) %>%
 mutate(bmi = as.numeric(bmi)) %>%
 na.omit()

```

```

corr_matrix <- cor(corr_data)
```

```

****Explanation:****

1. ****`select()`****: Keeps only numeric/binary columns
2. ****`mutate(bmi = as.numeric(bmi))`****: Converts BMI to numeric
 - Might have been stored as character/factor due to missing values
3. ****`na.omit()`****: Removes any rows with missing values
 - Correlation requires complete cases
4. ****`cor()`****: Computes Pearson correlation matrix
 - Returns matrix where each cell is correlation between two variables
 - Values range from -1 (perfect negative) to +1 (perfect positive)

****Visualization:****

```

```r
ggcorrplot(corr_matrix,
 method = "circle",
 type = "lower",
 lab = TRUE,
 lab_size = 3.5,
 colors = c("#2196F3", "white", "#F44336"))
```

```

- ****`method = "circle"`****: Circle size represents correlation magnitude
- ****`type = "lower"`****: Shows only lower triangle (avoids redundancy)
- ****`lab = TRUE`****: Displays correlation values in circles
- ****`colors`****: Gradient from blue (negative) through white (zero) to red (positive)

CHUNK 13: Remove Irrelevant Column (Lines 279-283)

```

```r
stroke_df <- stroke_raw %>% select(-id)
```

```

****Explanation:****

- ****`select(-id)`****: Keeps all columns EXCEPT 'id'
- The minus sign ****`-`**** means "exclude"
- ID columns are usually not useful for prediction (just row identifiers)

CHUNK 14: Handle Missing BMI (Lines ~284-310)

Note: This section appears to be truncated in the view, but typically handles missing BMI values by imputation

CHUNK 15: Feature Engineering & Encoding (Lines ~311-360)

Truncated section that likely handles:

- Converting categorical variables to numeric/factor format
- Creating dummy variables for categorical features
- Scaling numerical features if needed

CHUNK 16: Train-Test Split (Lines 451-458)

```r

set.seed(42)

```
train_index <- createDataPartition(stroke_df$stroke,
 p = 0.8,
 list = FALSE)
```

```
train_data <- stroke_df[train_index,]
```

```
test_data <- stroke_df[-train_index,]
```

```

****Explanation:****

- ****`createDataPartition()`**** (from ``caret``): Stratified data splitting
 - ``stroke_df$stroke``: The target variable
 - ``p = 0.8``: 80% training, 20% test split
 - ``list = FALSE``: Returns row indices as vector (not list)
 - ****Why stratified?****: Ensures both train and test sets have similar stroke/no-stroke proportions
- ****`train\_index`****: Vector of row numbers for training (e.g., `c(1, 3, 5, 8, ...)`)
- ****`[-train\_index]`****: Negative indexing selects rows NOT in training set

****Result:****

- Training set: 80% of data (used to train models)
- Test set: 20% of data (held out for evaluation)

CHUNK 17: Class Distribution Check (Lines ~452-458)

```r

```
cat("Training set:", nrow(train_data), "rows\n")
```

```
print(prop.table(table(train_data$stroke)))
\\`
```

**\*\*Explanation:\*\***

- `table()`: Counts observations per class
- `prop.table()`: Converts to proportions (0-1)
- Verifies that both classes are represented in training and test sets

---

## CHUNK 18: Handle Class Imbalance via ROSE (Lines 463-469)

```
\\`r
set.seed(42)
train_balanced <- ROSE(stroke ~ ., data = train_data, seed = 42)$data
\\`
```

**\*\*What is class imbalance?\*\***

- Stroke is rare (few positive cases, many negative cases)
- Models tend to predict "No Stroke" to minimize error
- This results in poor recall for actual stroke cases

**\*\*ROSE (Random Over-Sampling Examples):\*\***

- `stroke ~ .`: Formula (stroke predicted by all other variables)
- Randomly duplicates minority class and undersamples majority class
- Creates synthetic balanced dataset for training
- The `\$data` extracts just the balanced data (ROSE returns a list)

**\*\*Why only balance training data?\*\***

- Keep test data realistic (imbalanced) to assess real-world performance
- Train on balanced data to help model learn stroke patterns better

---

## CHUNK 19: Cross-Validation Setup (Lines 474-485)

```
\\`r
ctrl <- trainControl(
 method = "cv",
 number = 5,
 classProbs = TRUE,
 summaryFunction = twoClassSummary,
 savePredictions = "final"
)
\\`
```

**\*\*Explanation:\*\***

- **\*\*`trainControl()`\*\***: Configures model training parameters (from `caret`)
- **\*\*`method = "cv"`\*\***: Cross-Validation method
  - Divides training data into K folds
  - Trains K models, each leaving one fold out for validation

- Averages results for robust performance estimate
- `**`number = 5`**`: 5-fold cross-validation
  - 5 iterations, each with 4 folds training + 1 fold validation
- `**`classProbs = TRUE`**`: Computes class probabilities
  - Returns probability of belonging to each class
  - Needed for ROC curves and AUC calculation
- `**`summaryFunction = twoClassSummary`**`:
  - Computes ROC, Sensitivity, Specificity for binary classification
  - (Alternative: ``defaultSummary`` for accuracy, RMSE, etc.)
- `**`savePredictions = "final"`**`:
  - Saves predictions from final held-out fold
  - Useful for model diagnostics

---

## CHUNK 20: Helper Function - Evaluation Metrics (Lines 490-509)

```

`r
compute_metrics <- function(actual, predicted, positive_class = "Stroke")
{
 cm <- confusionMatrix(predicted, actual, positive = positive_class)
 acc <- cm$overall["Accuracy"]
 prec <- cm$byClass["Precision"]
 recall <- cm$byClass["Recall"]
 f1 <- cm$byClass["F1"]

 list(
 ConfusionMatrix = cm,
 Metrics = data.frame(
 Metric = c("Accuracy", "Precision", "Recall", "F1-Score"),
 Value = round(c(acc, prec, recall, f1), 4)
)
)
}
`r

```

**\*\*Function breakdown:\*\***

**\*\*Parameters:\*\***

- ``actual``: True class labels (from test set)
- ``predicted``: Model predictions
- ``positive_class = "Stroke"``: Which class is "positive" (default "Stroke")

**\*\*Inside the function:\*\***

1. **`**`confusionMatrix()`**`** (from ``caret``):
  - Compares predictions vs actual values
  - Creates a confusion matrix
  - Example:

```

 ...
 Predicted
 No Stroke Stroke
Actual No Stroke 1850 20
 Stroke 40 90
 ...

```

2. **\*\*Extracting metrics from confusion matrix:\*\***

- ``cm$overall["Accuracy"]``: Overall accuracy
- ``cm$byClass["Precision"]``: How many predicted strokes were correct
- ``cm$byClass["Recall"]``: How many actual strokes were caught
- ``cm$byClass["F1"]``: Harmonic mean of precision and recall

**\*\*Return value:\*\*** A list containing:

- ``ConfusionMatrix``: Full confusion matrix object
- ``Metrics``: Data frame with 4 metrics rounded to 4 decimal places

**\*\*Metric definitions:\*\***

- **\*\*Accuracy\*\***:  $(TP + TN) / \text{Total}$  - Overall correctness
- **\*\*Precision\*\***:  $TP / (TP + FP)$  - Of predicted strokes, how many are correct
- **\*\*Recall\*\***:  $TP / (TP + FN)$  - Of actual strokes, how many are caught
- **\*\*F1-Score\*\***:  $2 * (Precision * Recall) / (Precision + Recall)$  - Balance of precision and recall

---

## CHUNK 21: Model 1 - Logistic Regression (Lines 515-537)

```

` ``r
set.seed(42)
model_lr <- train(
 stroke ~ .,
 data = train_balanced,
 method = "glm",
 family = "binomial",
 trControl = ctrl,
 metric = "ROC"
)

pred_lr <- predict(model_lr, newdata = test_data)
results_lr <- compute_metrics(test_data$stroke, pred_lr)
` ``

```

**\*\*Explanation:\*\***

**\*\*Training:\*\***

- **\*\*`stroke ~ .`\*\***: Formula meaning "predict stroke using all other variables"
- **\*\*`method = "glm"`\*\***: Generalized Linear Model (logistic regression for binary)
- **\*\*`family = "binomial"`\*\***: Binary logistic regression family
- **\*\*`data = train\_balanced`\*\***: Use the balanced training data
- **\*\*`trControl = ctrl`\*\***: Use the 5-fold CV setup from earlier

```

- **`metric = "ROC"`**: Optimize based on ROC curve area

Prediction & Evaluation:
- **`predict(model_lr, newdata = test_data)`**: Applies trained model to
test set
 - Returns factor with levels "No Stroke" or "Stroke"
- **`compute_metrics()`**: Calls helper function to calculate all 4
metrics

Logistic Regression explanation:
- Estimates probability of stroke using logistic function
- Linear model transformed to 0-1 probability range
- Fast and interpretable
- Assumes linear relationship between features and log-odds of outcome

CHUNK 22: Model 2 - Random Forest (Lines 542-568)

```r
set.seed(42)
rf_grid <- expand.grid(mtry = c(3, 5, 7))

model_rf <- train(
  stroke ~ .,
  data = train_balanced,
  method = "rf",
  trControl = ctrl,
  tuneGrid = rf_grid,
  ntree = 200,
  metric = "ROC"
)

pred_rf <- predict(model_rf, newdata = test_data)
results_rf <- compute_metrics(test_data$stroke, pred_rf)
```

Hyperparameter tuning:

- **`expand.grid(mtry = c(3, 5, 7))`**:
 - Tests mtry = 3, 5, and 7
 - Creates 3 combinations (just one parameter)
 - Creates all possible combinations if multiple parameters

- **`mtry`**: Number of random features to consider at each split
 - Default for classification: sqrt(number_of_features)
 - Testing 3, 5, 7 to find optimal value

Training parameters:

- **`method = "rf"`**: Random Forest algorithm

- **`tuneGrid = rf_grid`**: Use the tuning grid above
 - Trains 3 different RF models (mtry=3, 5, 7)

```

- 5-fold CV for each, so 15 total RF models trained
- Selects best mtry value

- **ntree = 200**: Number of decision trees
  - Random Forest trains 200 separate trees
  - Final prediction averages predictions from all 200
  - More trees = more stable (but computationally expensive)

**Random Forest explanation:**

- Ensemble of 200 decision trees trained on random subsets
- Each tree splits on best feature (from random mtry candidates)
- Reduces overfitting through averaging and randomization
- Good for non-linear patterns

---

**CHUNK 23: Random Forest Feature Importance (Lines 573-592)**

```

```r
importance_df <- varImp(model_rf)$importance %>%
  rownames_to_column("Feature") %>%
  arrange(desc(Overall)) %>%
  head(15)
```

Explanation:

- varImp(model_rf): Extracts variable importance from Random Forest

- Measures how much each feature contributes to predictions
- Random Forest: Aggregates importance across all 200 trees
- Importance score based on how much each feature reduces impurity

- $importance: Extracts the importance data frame
- rownames_to_column("Feature"):
- Feature names are stored as row names
- Converts them to a column called "Feature"
- arrange(desc(Overall)):
- Sorts by Overall importance score
- desc() = descending (highest first)
- head(15): Keeps only top 15 features

Visualization:


```

```r
ggplot(importance_df, aes(x = reorder(Feature, Overall), y = Overall,
 fill = Overall)) +
 geom_bar(stat = "identity") +
 scale_fill_gradient(low = "#90CAF9", high = "#0D47A1") +
 coord_flip()
```

```


```

- **reorder(Feature, Overall)**: Reorders features by importance (x-axis)
- Makes bar chart sorted from least to most important
- **coord\_flip()**: Flips axes to make horizontal bar chart
- Easier to read long feature names

---

## CHUNK 24: Model 3 - Support Vector Machine (Lines 598-624)

```r

set.seed(42)

svm_grid <- expand.grid(C = c(0.5, 1, 2), sigma = c(0.01, 0.05))

```
model_svm <- train(
  stroke ~ .,
  data = train_balanced,
  method = "svmRadial",
  trControl = ctrl,
  tuneGrid = svm_grid,
  metric = "ROC"
)
```

```
pred_svm <- predict(model_svm, newdata = test_data)
results_svm <- compute_metrics(test_data$stroke, pred_svm)
```
```

**Hyperparameter tuning:**

- **expand.grid(C = c(0.5, 1, 2), sigma = c(0.01, 0.05))**:
  - Tests 3 values of C × 2 values of sigma = 6 combinations
  - Creates all possible combinations
- **C** (Cost parameter):
  - Controls misclassification tolerance
  - C = 0.5: More tolerance (larger margin, may misclassify some)
  - C = 2: Less tolerance (tight margin, try to classify all correctly)
  - Trade-off between margin size and training error
- **sigma** (RBF kernel parameter):
  - Controls shape of decision boundary
  - Small sigma = localized decision boundary (complex)
  - Large sigma = smooth decision boundary (simple)

**Training:**

- **method = "svmRadial"**: SVM with Radial Basis Function (RBF) kernel
  - RBF allows non-linear decision boundaries
  - Works well when classes not linearly separable

**SVM explanation:**

- Finds optimal hyperplane that maximizes margin between classes
- Can handle non-linear patterns through kernel tricks
- Works well for binary classification



```

- Sensitive to feature scaling (should normalize)

CHUNK 25: Comparative Metrics Table (Lines 629-659)

```r
comparison_table <- data.frame(
  Model = c("Logistic Regression", "Random Forest", "SVM (Radial)"),
  Accuracy = c(
    results_lr$Metrics$Value[1],
    results_rf$Metrics$Value[1],
    results_svm$Metrics$Value[1]
  ),
  Precision = c(
    results_lr$Metrics$Value[2],
    results_rf$Metrics$Value[2],
    results_svm$Metrics$Value[2]
  ),
  Recall = c(
    results_lr$Metrics$Value[3],
    results_rf$Metrics$Value[3],
    results_svm$Metrics$Value[3]
  ),
  F1_Score = c(
    results_lr$Metrics$Value[4],
    results_rf$Metrics$Value[4],
    results_svm$Metrics$Value[4]
  )
)
```

Explanation:

- Creates a comparison data frame with 3 rows (one per model) and 5 columns (Model + 4 metrics)

- **`results_*$Metrics$Value[1]`**: Extracts metric values
 - `*$Metrics` accesses metrics data frame
 - `*$Value` gets the numeric values column
 - `[1]` gets first element (Accuracy)
 - `[2]` gets second element (Precision), etc.

Table display:
```r
knitr::kable(comparison_table, caption = "Model Comparison Table")
View(comparison_table)
```

- **`knitr::kable()`**: Formats as HTML table
- **`View()`**: Opens interactive viewer in RStudio

```

```
CHUNK 26: ROC Curve Probability Computation (Lines 663-674)
```

```
```r
true_labels <- ifelse(test_data$stroke == "Stroke", 1, 0)

prob_lr <- predict(model_lr, test_data, type = "prob")[, "Stroke"]
prob_rf <- predict(model_rf, test_data, type = "prob")[, "Stroke"]
prob_svm <- predict(model_svm, test_data, type = "prob")[, "Stroke"]

roc_lr <- pROC::roc(true_labels, prob_lr)
roc_rf <- pROC::roc(true_labels, prob_rf)
roc_svm <- pROC::roc(true_labels, prob_svm)
```
```

```
Explanation:
```

1. **\*\*Convert target to numeric (0/1):\*\***
  - ``ifelse(test_data$stroke == "Stroke", 1, 0)``
  - Needed because pROC expects numeric labels
2. **\*\*Extract probabilities:\*\***
  - **`**`predict(..., type = "prob")`**`**: Returns probability matrix
    - Output matrix:

|     | No Stroke | Stroke |
|-----|-----------|--------|
| 1   | 0.85      | 0.15   |
| 2   | 0.45      | 0.55   |
| ... |           |        |
  - **`**[, "Stroke"]`**`**: Extracts probability of "Stroke" class only
    - Result: `c(0.15, 0.55, ...)`
3. **\*\*Compute ROC objects:\*\***
  - **`**`pROC::roc(true_labels, prob_svm)`**`** (from ``pROC`` package)
  - Compares true labels vs predicted probabilities
  - Computes True Positive Rate and False Positive Rate at different thresholds
  - Stores TPR, FPR, and AUC in ROC object

```

```

```
CHUNK 27: ROC Dataframe Binding (Lines 678-701)
```

```
```r
build_roc_df <- function(roc_obj, model_name){
  data.frame(
    TPR = roc_obj$sensitivities,
    FPR = 1 - roc_obj$specificities,
    Model = model_name
  )
}
```

```

roc_df <- dplyr::bind_rows(
  build_roc_df(roc_lr, "Logistic Regression"),
  build_roc_df(roc_rf, "Random Forest"),
  build_roc_df(roc_svm, "SVM")
)
...

**Explanation:**

**Helper function:**
- **`roc_obj$sensitivities`**: True Positive Rate (TPR)
  - Also called "Recall"
  -  $TP / (TP + FN)$  = proportion of actual strokes detected
- **`1 - roc_obj$specificities`**: False Positive Rate (FPR)
  -  $1 - specificity = FP / (FP + TN)$ 
  - Proportion of non-strokes incorrectly classified as stroke

**ROC Curve concept:**
- ROC = Receiver Operating Characteristic
- Plots TPR (y-axis) vs FPR (x-axis) at different decision thresholds
- Perfect classifier: Curve goes straight up then right (AUC = 1.0)
- Random classifier: Diagonal line (AUC = 0.5)

**Binding dataframes:**
```r
roc_df <- dplyr::bind_rows(
 build_roc_df(roc_lr, "Logistic Regression"),
 build_roc_df(roc_rf, "Random Forest"),
 build_roc_df(roc_svm, "SVM")
)
...

- **`bind_rows()`**: Vertically stacks data frames
- Creates one tall data frame with data from all 3 models
- `Model` column identifies which model each row belongs to

Result structure:
...
 TPR FPR Model
1 1.00 1.00000000 Logistic Regression
2 1.00 0.99827586 Logistic Regression
3 1.00 0.99654676 Logistic Regression
...

CHUNK 28: ROC Curve Visualization (Lines 694-701)

```r
ggplot(roc_df, aes(x = FPR, y = TPR, color = Model)) +
  geom_line(size = 1.2) +
  scale_color_manual(values = c("#1976D2", "#388E3C", "#F57C00")) +

```

```

geom_abline(linetype = "dashed") +
labs(title = "ROC Curve Comparison",
      x = "False Positive Rate",
      y = "True Positive Rate") +
theme_minimal()
```

Explanation:

- **`geom_line(size = 1.2)`**: Draws curves for each model
 - `size = 1.2`: Line thickness

- **`scale_color_manual()`**: Assigns colors to models
 - Blue (#1976D2), Green (#388E3C), Orange (#F57C00)

- **`geom_abline()`**: Reference diagonal line
 - Represents random classifier (AUC = 0.5)
 - `linetype = "dashed"`: Dashed line style

Interpretation:

- Curve closer to top-left = better model
- Model with highest AUC is best
- Trade-off between catching strokes (TPR) and false alarms (FPR)

CHUNK 29: Confusion Matrix Visualization (Lines 705-722)

```r
plot_cm <- function(actual, predicted, title){
  cm <- table(actual, predicted)
  df <- as.data.frame(cm)

  ggplot(df, aes(x = predicted, y = actual, fill = Freq)) +
    geom_tile() +
    geom_text(aes(label = Freq), color = "white", size = 5) +
    labs(title = title, x = "Predicted", y = "Actual") +
    theme_minimal()
}

cm1 <- plot_cm(test_data$stroke, pred_lr, "Logistic Regression")
cm2 <- plot_cm(test_data$stroke, pred_rf, "Random Forest")
cm3 <- plot_cm(test_data$stroke, pred_svm, "SVM (Radial)")

gridExtra::grid.arrange(cm1, cm2, cm3, ncol = 3)
```

Explanation:

Helper function:

1. **`table(actual, predicted)`**: Cross-tabulation
  ```
  Predicted
  
```

	No Stroke	Stroke
Actual No Stroke	1850	20
Stroke	40	90

...

2. `**`as.data.frame(cm)`**`: Converts to long format

...

	actual	predicted	Freq
1	No Stroke	No Stroke	1850
2	Stroke	No Stroke	40
3	No Stroke	Stroke	20
4	Stroke	Stroke	90

...

****Visualization:****

- `**`geom_tile()`**`: Creates heatmap with colored squares
 - Color intensity represents frequency (Freq)
- `**`geom_text()`**`: Adds frequency counts inside tiles
 - ``color = "white"``: White text for contrast

****Interpretation:****

- Diagonal cells (top-left, bottom-right): Correct predictions
- Off-diagonal cells: Errors
- Can compare error patterns between models

CHUNK 30: AUC Summary Table (Lines 726-734)

```

```r
auc_df <- data.frame(
 Model = c("Logistic Regression", "Random Forest", "SVM (Radial)"),
 AUC = round(c(pROC::auc(roc_lr), pROC::auc(roc_rf),
pROC::auc(roc_svm)), 4)
)

```

```

knitr::kable(auc_df, caption = "AUC Comparison")
View(auc_df)
```

```

****Explanation:****

- `**`pROC::auc(roc_obj)`**`: Extracts AUC from ROC object
 - AUC = Area Under Curve
 - Ranges from 0.5 (random) to 1.0 (perfect)
 - Single number summarizing model's discrimination ability
- `**`round(..., 4)`**`: Rounds to 4 decimal places
- ****Output table****: Shows AUC score for each model side-by-side
 - Allows easy comparison of which model performs best
 - Higher AUC = better model

```
**AUC Interpretation:**
- AUC = 0.9-1.0: Excellent
- AUC = 0.8-0.9: Good
- AUC = 0.7-0.8: Fair
- AUC = 0.6-0.7: Poor
- AUC = 0.5: Useless (like random guessing)
```

🇮🇹 Key Concepts Summary

Machine Learning Pipeline

1. ****Data Collection****: Load from Google Drive
2. ****EDA****: Visualize and understand data
3. ****Preprocessing****: Handle missing values, encode categorical variables
4. ****Train-Test Split****: 80-20 split with stratification
5. ****Class Imbalance Handling****: ROSE oversampling
6. ****Model Training****: Train 3 different algorithms
7. ****Evaluation****: Compare using 4 metrics and ROC curves

Models Trained

1. ****Logistic Regression****: Fast baseline, linear boundaries
2. ****Random Forest****: Ensemble of 200 trees, handles non-linear patterns
3. ****Support Vector Machine****: Finds optimal hyperplane, non-linear kernel

Evaluation Metrics

- ****Accuracy****: Overall correctness (misleading with imbalanced data)
- ****Precision****: False alarm rate (of predicted strokes, how many correct)
- ****Recall****: Detection rate (of actual strokes, how many caught)
- ****F1-Score****: Balance between precision and recall
- ****AUC****: Overall ranking ability (area under ROC curve)

Why This Approach?

- Multiple models compare different algorithms
- 5-fold CV provides robust evaluation
- ROC curves and AUC focus on discrimination ability
- ROSE handles class imbalance (strokes are rare)
- Balanced training set but imbalanced test set reflects real-world distribution

🎯 Conclusion

This Rmd file demonstrates a complete machine learning workflow for stroke prediction. It compares three classification algorithms using rigorous evaluation methodology, handles class imbalance appropriately, and produces comprehensive visualizations and comparative tables to support decision-making about which model to deploy.